

Parking System Classes: Write Up

ICT 4305: Object Oriented Methods and Programming

for

Master of Science

Information Technology

Kalika Browder

University of Denver College of Professional Studies

October 5, 2025

Faculty: Nirav Shah, M.S. , MBA

Director: Cathie Wilson, M.S.

Dean: Michael J. McGuire, MLS

Reflection

This assignment was both incredibly beneficial and incredibly frustrating for me. I find certain aspects of object-oriented design so straightforward, and I find other parts very meticulous. The creation of certain classes such as Car, Customer, ParkingLot, Address, and CarType was relatively easy for me because the instructions provided were clear with attribute guidance. Furthermore, defining constructors, getters, and toString() methods followed the standard Java patterns that we've been learning about through course materials, and the relationship between classes (like a customer owning multiple cars), mapped naturally to lists and aggregation.

The more challenging parts of this assignment involved me understanding how the classes needed to interact, especially for the ParkingLot class and its overall handling of parked cars. Deciding how to store cars, manage the capacity, and represent entry logic without implementing a full billing system really tested me to think abstractly. I needed to ensure the proper use of the CarType enum to reflect compact vs. SUV cars for potential discount logic, which required thorough attention to detail. Also, I had to realize that Java cannot run a class without a Main method (which I had forgotten from my previous Java course), but creating a dedicated class and also utilizing the JUnit tests helped me eventually resolve the issues I was experiencing.

I feel what helped me most throughout this assignment was breaking the assignment into modular tasks. The first thing I did was implement the data model, then the relationships, and then move to the testing phase. I experienced quite a bit of debugging, as well. Using JUnit

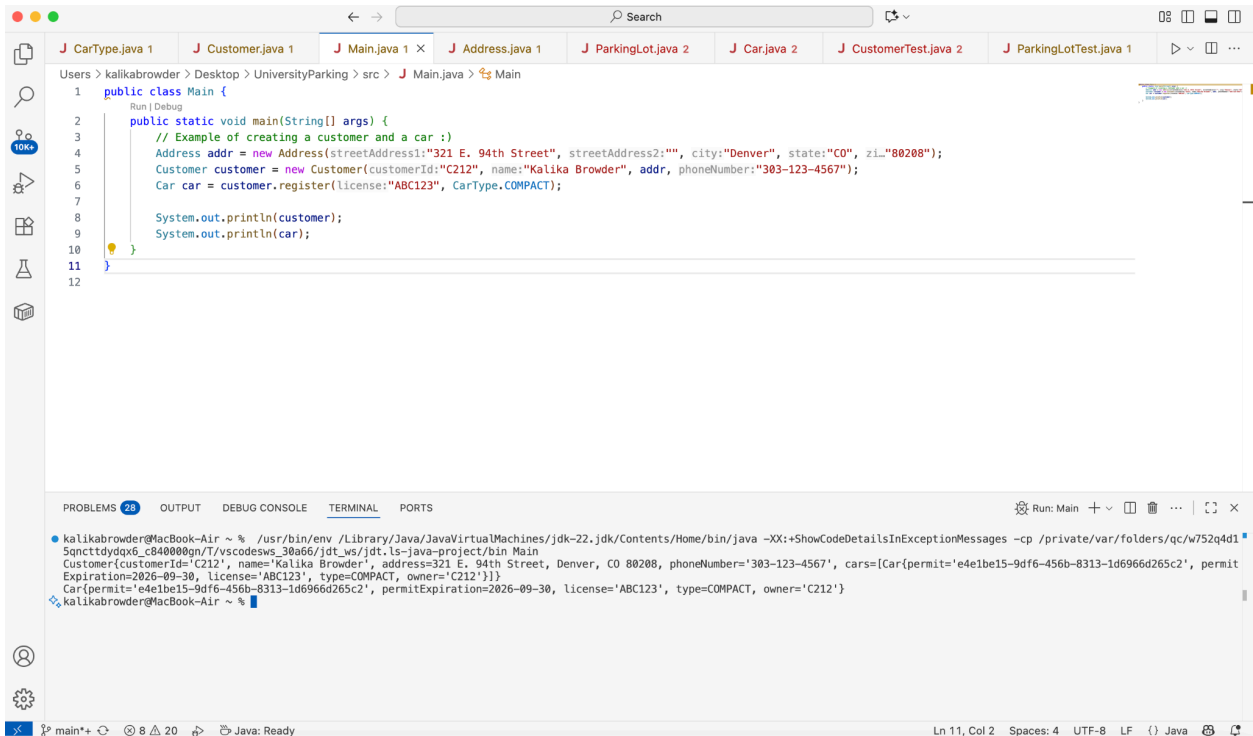
for testing reinforced good development practices by separating test logic from main code. Accessing documentation for `java.time.LocalDate` and understanding how to auto-generate permits were also so essential for me to implement a functional Car class that actually made sense in this scenario.

I wish I also remembered that each Java class must reside in its own file named after the class (which I eventually did recall). This understanding was obviously crucial to the final product. Additionally, prior familiarity with UML diagrams did make the design phase faster, but I'm still feeling like it's not my strong suite. However, translating class relationships into diagrams helped solidify my understanding of ownership and multiplicity.

A lot of my implementation decisions were guided by clarity, modularity, and real life application/representation. For example, storing multiple cars per customer as `List<Car>` allows scalability. Representing `CarType` as an enum ensures type safety and also enforces valid car types presented within the scenario. `Parkinglot`'s `entry()` method focuses on capacity and parking logic without also prematurely implementing billing. The constructors were designed to automatically generate permits and set expiration dates, which I felt the need to implement for a realistic touch. All methods and attributes were kept private with public getters to enforce encapsulation.

Below are screenshots representing both the successful compilation of all Java classes, as well as the execution of JUnit tests which confirm the expected behavior. Overall, this approach helped my understanding of Java, the principles we have been learning about in object-oriented design, and reinforced professional software development practices, so I'm

grateful to have learned more from this assignment, and hope to only continue to grow throughout this process in the future.



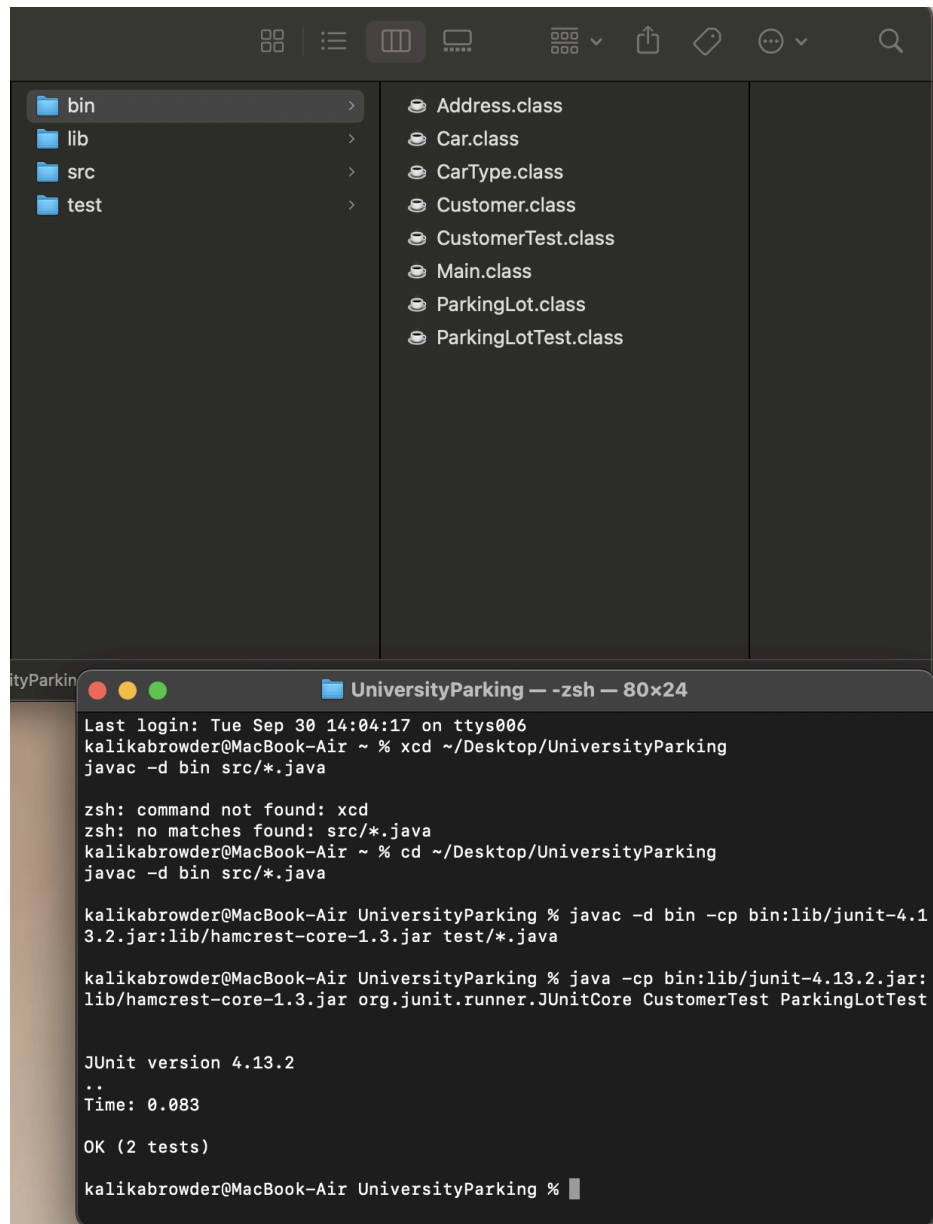
The screenshot shows an IDE with a project named 'UniversityParking'. The 'Main.java' file is open, displaying the following code:

```
1 public class Main {
2     // Run | Debug
3     public static void main(String[] args) {
4         // Example of creating a customer and a car :)
5         Address addr = new Address(streetAddress1:"321 E. 94th Street", streetAddress2:"", city:"Denver", state:"CO", zip:"80208");
6         Customer customer = new Customer(customerId:"C212", name:"Kalika Browder", addr, phoneNumber:"303-123-4567");
7         Car car = customer.register(license:"ABC123", CarType.COMPACT);
8
9         System.out.println(customer);
10        System.out.println(car);
11    }
12 }
```

The terminal output shows the execution of the program:

```
kalikabrowder@MacBook-Air ~ % /usr/bin/env /Library/Java/JavaVirtualMachines/jdk-22.jdk/Contents/Home/bin/java -XX:+ShowCodeDetailsInExceptionMessages -cp /private/var/folders/qc/w752q4d1-5qncctdydx6_c840000gn/T/vscodesws_30a66/jdt_ws/jdt.ls-java-project/bin Main
Customer{customerId='C212', name='Kalika Browder', address=321 E. 94th Street, Denver, CO 80208, phoneNumber='303-123-4567', cars=[Car{permit='e4e1be15-9df6-456b-8313-1d6966d265c2', permitExpiration=2026-09-30, license='ABC123', type=COMPACT, owner='C212'}]}
Car{permit='e4e1be15-9df6-456b-8313-1d6966d265c2', permitExpiration=2026-09-30, license='ABC123', type=COMPACT, owner='C212'}
```

Output of Main.java class.



Output of CustomerTest.java + ParkingLotTest class using JUnit Testing.